



**UNIVERSIDAD
DE GRANADA**

Facultad de Ciencias

Grado en Matemáticas

Estadística Computacional

Trabajo en Grupo:

Modelos de Clasificación con el paquete CARET

Autores:

Moisés Jiménez Fernández

Pablo Martín Palomino

Natalia Yue Gallardo Pretel

Mario Líndez Martínez

Pablo Reyes Sousa

Diego Ortega Sánchez

Curso 2025/26

Índice

1. Introducción y Planteamiento del Problema	2
2. Preparación y Exploración de los Datos	3
3. Benchmarking: Entrenamiento y Evaluación de Modelos	4
3.1. Regresión Logística Multinomial	4
Construcción y entrenamiento del modelo	4
Evaluación del modelo	5
Análisis de los resultados	6
3.2 Random Forest	8
Construcción y entrenamiento del modelo	8
Evaluación del modelo	9
Análisis de resultados	10
3.3. K-Nearest Neighbors (KNN)	11
Construcción y entrenamiento del modelo	11
Evaluación del modelo	12
Análisis de los resultados	14
3.4. Máquinas de Vector de Soporte (SVM) con Kernel Radial	15
Construcción y entrenamiento del modelo	15
Evaluación del modelo	16
Análisis de los resultados	16
3.5. Gradient Boosting	17
Construcción y entrenamiento del modelo	17
Evaluación del modelo	18
Análisis de los resultados	19
3.6. Red Neuronal (Multilayer Perceptron)	21
Análisis de los resultados	22
4. Análisis Visual Comparativo de Fronteras de Decisión	24
5. Discusión y Conclusiones	27

1. Introducción y Planteamiento del Problema

El objetivo de este documento es abordar un problema clásico de clasificación desde la perspectiva del Aprendizaje Automático (*Machine Learning*), ilustrando cómo el ecosistema de R proporciona herramientas robustas y unificadas para la modelización predictiva.

Para este estudio, utilizaremos el conjunto de datos *iris*. Este *dataset* multivariante contiene 150 observaciones correspondientes a tres especies de flores de la familia Iris (*Iris setosa*, *Iris versicolor* e *Iris virginica*). Para cada flor, se disponen de cuatro características morfológicas medidas en centímetros: la longitud y la anchura de los sépalos y pétalos.

Desde un punto de vista matemático y computacional, nos enfrentamos a un problema de clasificación supervisada donde buscamos aproximar una función $f : \mathbb{R}^4 \rightarrow \{C_1, C_2, C_3\}$ que mapee las características geométricas al espacio de clases de las especies.

En lugar de depender de un único enfoque metodológico, esta *vignette* explora el uso del paquete *caret* (*Classification And Regression Training*). Este paquete actúa como una interfaz unificada que simplifica drásticamente el proceso de entrenamiento, ajuste de hiperparámetros y evaluación de múltiples modelos predictivos. A lo largo del documento, contrastaremos enfoques que van desde fronteras de decisión puramente lineales (Regresión Logística) hasta mapeos no lineales complejos (Redes Neuronales y Gradient Boosting), analizando su viabilidad, interpretabilidad y rendimiento empírico.

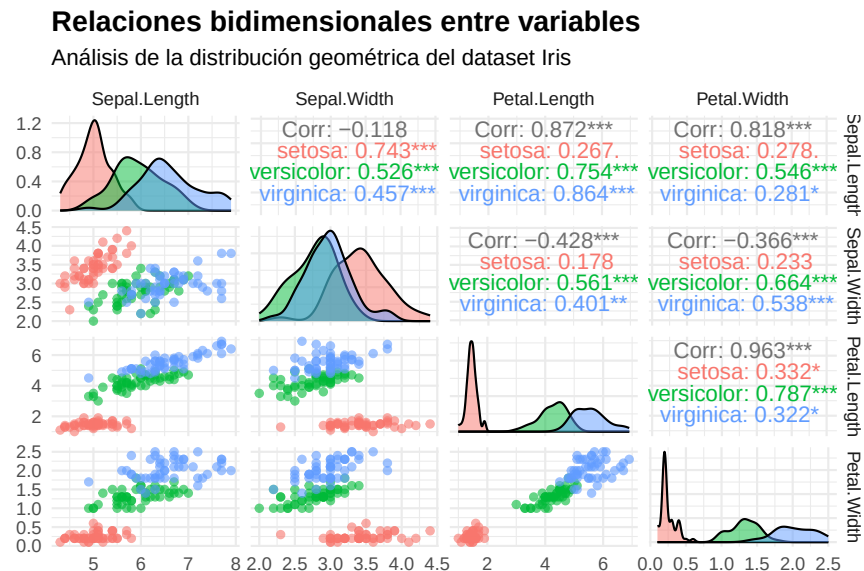
2. Preparación y Exploración de los Datos

Para garantizar una comparativa rigurosa entre los distintos modelos, realizaremos una única partición del conjunto de datos. Reservaremos un 80% de las observaciones para la fase de entrenamiento y un 20% para la fase de test (evaluación real).

Antes de modelar, es útil visualizar cómo se distribuyen las variables geométricas según la especie.

```
library(ggplot2)
#install.packages("GGally")
library(GGally)

# Matriz de dispersión detallada (pairwise scatter plots)
ggpairs(iris,
  columns = 1:4,          # Solo las variables geométricas
  aes(color = Species),   # Colorear por especie
  upper = list(continuous = wrap("cor", size = 4)), # Correlación
  lower = list(continuous = wrap("points", alpha = 0.6, size = 1.5)), # Dispersión
  diag = list(continuous = wrap("densityDiag", alpha = 0.5))) + # Densidad
  theme_minimal() +
  labs(title = "Relaciones bidimensionales entre variables",
    subtitle = "Análisis de la distribución geométrica del dataset Iris") +
  theme(plot.title = element_text(face = "bold", size = 14))
```



3. Benchmarking: Entrenamiento y Evaluación de Modelos

3.1. Regresión Logística Multinomial

La regresión logística multinomial es una extensión natural de la regresión logística binaria para abordar problemas de clasificación con más de dos categorías. En este caso, la variable de respuesta es *Species*, que puede tomar tres valores: *setosa*, *versicolor* y *virginica*, por lo que el modelo debe estimar la probabilidad de pertenencia a cada una de estas clases.

A diferencia de otros métodos más flexibles, la regresión logística multinomial construye fronteras de decisión lineales en el espacio de predictores. Para ello, combina linealmente las variables explicativas y transforma los resultados mediante la función `softmax`.

De forma general, para una observación con vector de características x , la probabilidad de pertenecer a la clase k se expresa como:

$$P(Y = k \mid X = x) = \frac{e^{\beta_{k0} + \beta_k^T x}}{\sum_{j=1}^K e^{\beta_{j0} + \beta_j^T x}}, \quad k = 1, \dots, K.$$

En nuestro caso, $K = 3$, ya que existen tres especies posibles. El modelo asigna cada observación a la clase con mayor probabilidad estimada.

Construcción y entrenamiento del modelo

Para mantener la misma metodología común del documento, el modelo se ajusta mediante la función `train()` de `caret`, utilizando el método "multinom" y una malla de valores para el parámetro de regularización `decay`.

```
set.seed(123)

# Malla de hiperparámetros
multinomGrid <- expand.grid(
  decay = c(0.1, 0.01, 0.001, 0) # Parámetro de penalización L2
)

# Entrenamiento del modelo de regresión logística multinomial
mod_logit <- train(
  Species ~ .,
  data = trainData,
  method = "multinom",
  tuneGrid = multinomGrid,
  trace = FALSE
)

# Hiperparámetro óptimo seleccionado por validación cruzada
mod_logit$bestTune
#> decay
#> 4 0.1

# Resumen del modelo final
#summary(mod_logit$finalModel)
```

Evaluación del modelo

Una vez entrenado el modelo, se realizan predicciones tanto sobre el conjunto de entrenamiento como sobre el conjunto de test. La evaluación en entrenamiento permite analizar el grado de ajuste del modelo a los datos utilizados durante la estimación de los parámetros. Sin embargo, esta medida puede ser optimista, ya que el modelo ya ha visto esas observaciones.

Por ello, se complementa con la evaluación sobre el conjunto de test, formado por observaciones no utilizadas durante el entrenamiento. La comparación entre ambas matrices de confusión permite detectar posibles indicios de sobreajuste: si el rendimiento en entrenamiento fuera muy superior al obtenido en test, podría indicar que el modelo se ha ajustado demasiado a los datos de entrenamiento. En cambio, resultados similares en ambos conjuntos sugieren una buena capacidad de generalización.

```
# Predicción sobre el conjunto de entrenamiento
pred_logit_train <- predict(mod_logit, newdata = trainData)

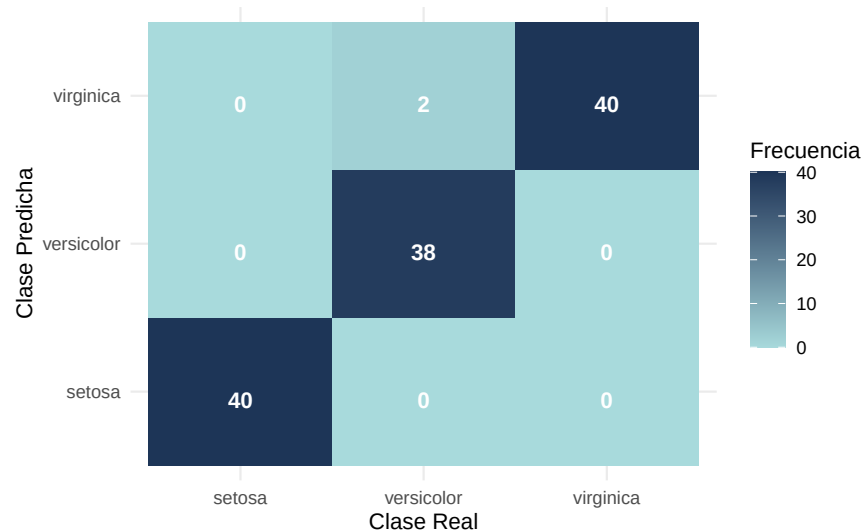
# Matriz de confusión sobre entrenamiento
cm_logit_train <- confusionMatrix(pred_logit_train, trainData$Species)

# Mostrar resultados de entrenamiento
#cm_logit_train

# Función modular para dibujar matrices de confusión
plot_cm <- function(cm, model_name, type = "Entrenamiento") {
  cm_data <- as.data.frame(cm$table)
  ggplot(data = cm_data, aes(x = Reference, y = Prediction, fill = Freq)) +
    geom_tile() +
    geom_text(aes(label = Freq), vjust = 1, color = "white", fontface = "bold") +
    scale_fill_gradient(low = "#a8dadc", high = "#1d3557") +
    theme_minimal() +
    labs(title = paste("Matriz de Confusión en", type, ":", model_name),
         x = "Clase Real",
         y = "Clase Predicha",
         fill = "Frecuencia") +
    theme(plot.title = element_text(hjust = 0.5, face = "bold"))
}

plot_cm(cm_logit_train, "Regresión Logística", "Entrenamiento")
```

Matriz de Confusión en Entrenamiento : Regresión Logística



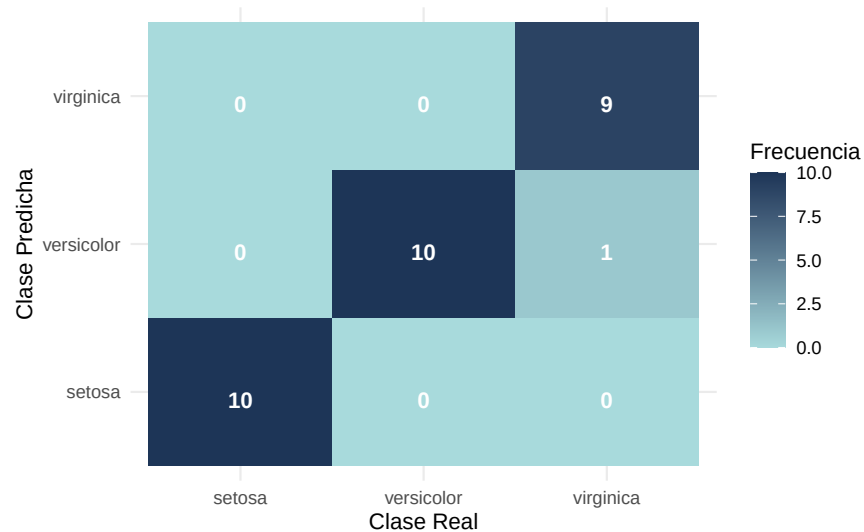
```
# Predicción sobre el conjunto de test
pred_logit_test <- predict(mod_logit, newdata = testData)

# Matriz de confusión sobre test
cm_logit_test <- confusionMatrix(pred_logit_test, testData$Species)

# Mostrar resultados de test
#cm_logit_test

plot_cm(cm_logit_test, "Regresión Logística", "Test")
```

Matriz de Confusión en Test : Regresión Logística



Análisis de los resultados

Los resultados muestran que la regresión logística multinomial obtiene un rendimiento muy alto tanto en entrenamiento como en test. En el conjunto de entrenamiento clasifica correctamente 118 de las 120

observaciones, alcanzando una exactitud aproximada del 98,33%. Los únicos errores corresponden a 2 observaciones reales de *versicolor* que son clasificadas como *virginica*.

En el conjunto de test, el modelo mantiene una exactitud elevada, con 29 aciertos sobre 30 observaciones, es decir, un 96,67%. El único error se produce entre las especies *virginica* y *versicolor*, lo que resulta coherente con el mayor solapamiento morfológico entre ambas clases. En cambio, la especie *setosa* se clasifica correctamente en todos los casos, debido a su clara separación respecto al resto.

La comparación entre entrenamiento y test no muestra indicios claros de sobreajuste, ya que la diferencia de rendimiento entre ambos conjuntos es pequeña. Esto sugiere que el modelo generaliza adecuadamente sobre observaciones no vistas.

La frontera de decisión bidimensional confirma esta interpretación. En el plano formado por *Petal.Length* y *Petal.Width*, la región de *setosa* aparece claramente separada, mientras que *versicolor* y *virginica* se sitúan en zonas más próximas. Además, la forma aproximadamente lineal de las fronteras refleja la naturaleza del modelo. A pesar de su simplicidad, la regresión logística multinomial consigue una clasificación muy precisa, por lo que constituye un modelo base sólido, interpretable y computacionalmente eficiente para este problema.

3.2 Random Forest

El método de random forest es un método ensemble basado en el entrenamiento de un gran número de árboles de decisión independientes (en tarea de clasificación) utilizando diferentes muestras aleatorias sacadas con reemplazo y el bosque devuelve la característica más votada (la moda) de entre todos los árboles.

A la hora de crear un árbol de decisión, lo normal es coger la variable que permite diferenciar mejor las instancias de las clases. Sin embargo, si utilizamos esta perspectiva para crear los diferentes árboles del bosque, se heredarían los errores obtenidos utilizando esta forma de división. Por esta razón, random forest incluye un hiperparámetro: el número de características a tener en cuenta en cada árbol, las cuales se elegirán de forma aleatoria a la hora de construir los árboles. A este parámetro lo llamaremos $mtry$ y, como el dataset tiene 4 características, $mtry \in \{1, 2, 3, 4\}$.

Una vez creado un bosque con S árboles, dada una nueva instancia x , el método ejecuta la entrada en cada uno de los árboles $\hat{f}_i(x)$ y devuelve el valor más frecuente. Es decir:

$$\hat{f}(x) = \text{moda}\{\hat{f}_1(x), \hat{f}_2(x), \dots, \hat{f}_S(x)\}$$

Uno podría pensar que el parámetro S (número de árboles del bosque) es otro hiperparámetro: si es muy pequeño, hay subajuste; si es muy grande, habrá sobreajuste (como ocurre en la mayoría de modelos). Sin embargo, en el artículo introductorio de los random forest (*Random Forests* de Leo Breiman) se demuestra que a mayor S , mejor desempeño tendrá el bosque con nuevas muestras. La demostración de este suceso se debe a la Ley de los Grandes Números.

Construcción y entrenamiento del modelo

Para entrenar el modelo se usa `train` del paquete `Caret` utilizando como parámetro `method = "rf"`. Se define una malla de valores para el hiperparámetro `mtry` y se selecciona el mejor valor mediante validación cruzada de 10 particiones.

```
set.seed(123)

S <- 1000

# Cross validation
ctrl_rf <- trainControl(method = "cv", number = 10)

# Grid de hiperparámetros
rfGrid <- expand.grid(mtry = 1:4)

# Entrenamiento
mod_rf <- train(
  Species ~ .,
  data = trainData,
  method = "rf",
  tuneGrid = rfGrid,
  trControl = ctrl_rf,
  ntree = S,
)

# Valor óptimo de mtry seleccionado por validación cruzada
print(paste("Valor óptimo de mtry seleccionado: ", mod_rf$bestTune))
#> [1] "Valor óptimo de mtry seleccionado: 1"
```

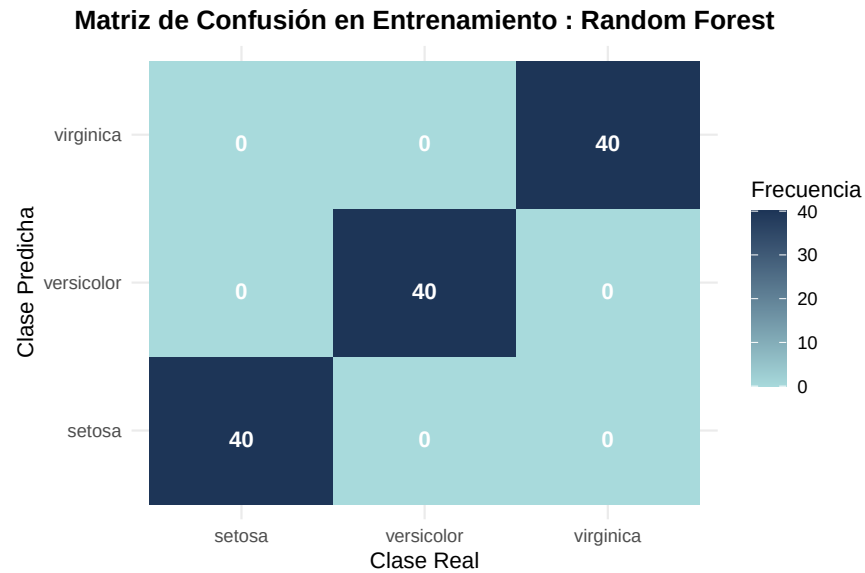
Evaluación del modelo

Al igual que en los modelos anteriores, evaluamos el rendimiento tanto sobre el conjunto de entrenamiento como sobre el conjunto de test.

```
# Predicción entrenamiento
pred_rf_train <- predict(mod_rf, newdata = trainData)

# Matriz de confusión de entrenamiento
cm_rf_train <- confusionMatrix(pred_rf_train, trainData$Species)

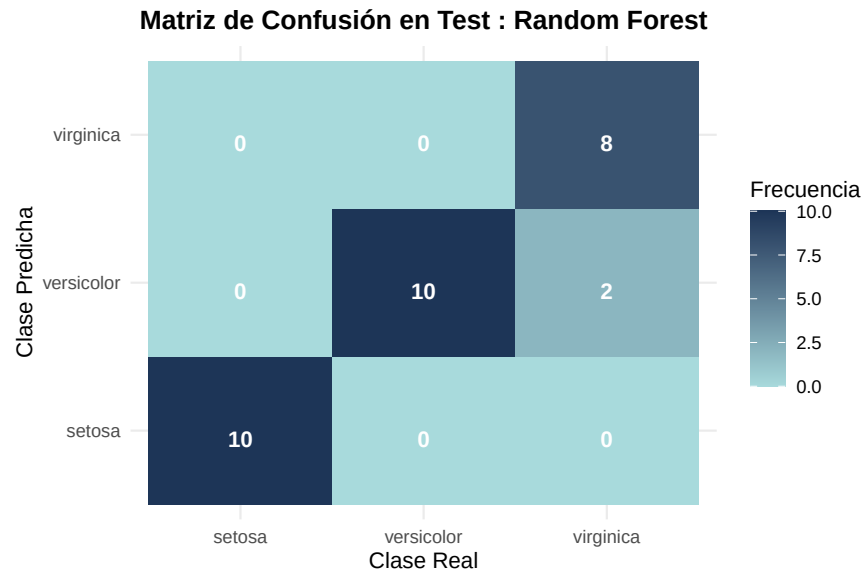
plot_cm(cm_rf_train, "Random Forest", "Entrenamiento")
```



```
# Predicción test
pred_rf_test <- predict(mod_rf, newdata = testData)

# Matriz de confusión de test
cm_rf_test <- confusionMatrix(pred_rf_test, testData$Species)

plot_cm(cm_rf_test, "Random Forest", "Test")
```



Las matrices de confusión muestran que el modelo de Random Forest tuvo un rendimiento perfecto en entrenamiento y uno bastante bueno en test. La precisión en entrenamiento es del 100% mientras que en test acertó 28 de las 30 muestras, con una precisión del 93.3%.

La clase setosa es perfectamente separada (se puede observar que es la diferente en las observaciones geométricas) mientras que hay algo de confusión entre la clase virgínica y la versicolor, que hay un poco más de solapamiento en sus distribuciones.

Como se ve, el modelo funciona bastante bien y, debido al ser un ensemble, no existe preocupación por posibles sobreajustes.

Análisis de resultados

En general, se observa que Random Forest produce muy buenos resultados para el problema de clasificación. Posee una matriz de confusión perfecta para entrenamiento y una muy buena para test. Además, la base teórica de la que se deduce imposibilidad de sobreajuste puede permitirnos aumentar el número de árboles sin miedo alguno, pudiendo mejorar el modelo aún más. Sin embargo, si bien es cierto que en este problema tenemos pocas características y, por tanto, la creación de árboles es mucho más rápida, en problemas de clasificación en los que hay cientos de características, la creación de los árboles puede suponer un gran coste temporal, por lo que cabría razonar la idea de perder precisión de un modelo a cambio de ganar en optimización.

3.3. K-Nearest Neighbors (KNN)

El método **K-Nearest Neighbors** (KNN) es un algoritmo de clasificación supervisada no paramétrico. A diferencia de otros modelos, KNN no estima explícitamente una función de decisión durante una fase de entrenamiento tradicional. En su lugar, conserva las observaciones del conjunto de entrenamiento y clasifica cada nueva observación comparándola con los datos ya conocidos.

En nuestro caso, si una flor tiene medidas parecidas a las de ciertas flores del conjunto de entrenamiento, es razonable asignarle la misma especie que predomina entre esas flores cercanas. Para medir esta cercanía se utiliza habitualmente la distancia euclídea. Si tenemos dos observaciones x_i y x_j con p variables predictoras, dicha distancia viene dada por:

$$d(x_i, x_j) = \sqrt{\sum_{l=1}^p (x_{il} - x_{jl})^2}.$$

Dada una nueva observación x , el modelo busca sus k vecinos más cercanos y asigna como predicción la clase mayoritaria entre ellos:

$$\hat{y}(x) = \text{moda}\{y_i : x_i \in N_k(x)\},$$

donde $N_k(x)$ representa el conjunto de los k vecinos más próximos a x .

Por tanto, el hiperparámetro fundamental del modelo es el número de vecinos k . Valores pequeños de k generan fronteras de decisión muy flexibles y sensibles al ruido, por lo que pueden producir sobreajuste. En cambio, valores grandes de k suavizan la frontera de decisión, aunque si son excesivos pueden provocar infraajuste.

Además, KNN es especialmente sensible a la escala de las variables, ya que se basa directamente en distancias. Por esta razón, antes de aplicar el algoritmo se estandarizan las variables predictoras mediante centrado y escalado.

Construcción y entrenamiento del modelo

Para entrenar el modelo se utiliza de nuevo la función `train()` del paquete `caret`. Se define una malla de valores impares para k , evitando empates frecuentes en la votación de los vecinos, y se selecciona el mejor valor mediante validación cruzada de 10 particiones.

```
set.seed(123)

# Control de validación cruzada
ctrl_knn <- trainControl(method = "cv", number = 10)

# Malla de hiperparámetros: número de vecinos
knnGrid <- expand.grid(
  k = seq(1, 25, by = 2)
)

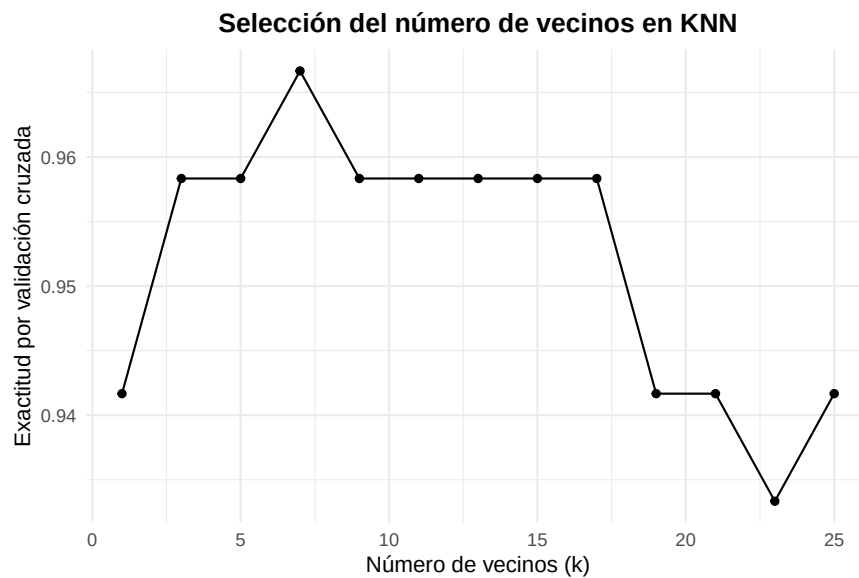
# Entrenamiento del modelo KNN
mod_knn <- train(
  Species ~ .,
  data = trainData,
  method = "knn",
  preProcess = c("center", "scale"),
  tuneGrid = knnGrid,
  trControl = ctrl_knn
```

```
)

# Valor óptimo de k seleccionado por validación cruzada
print(paste("Valor óptimo de k seleccionado: ", mod_knn$bestTune$k))
#> [1] "Valor óptimo de k seleccionado: 7"
```

Podemos representar cómo varía la exactitud estimada por validación cruzada en función del número de vecinos considerado.

```
ggplot(mod_knn) +
  theme_minimal() +
  labs(title = "Selección del número de vecinos en KNN",
       x = "Número de vecinos (k)",
       y = "Exactitud por validación cruzada") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
```



Confirmamos visualmente lo devuelto por el proceso de validación cruzada, siendo el número óptimo de vecinos de $k = 7$

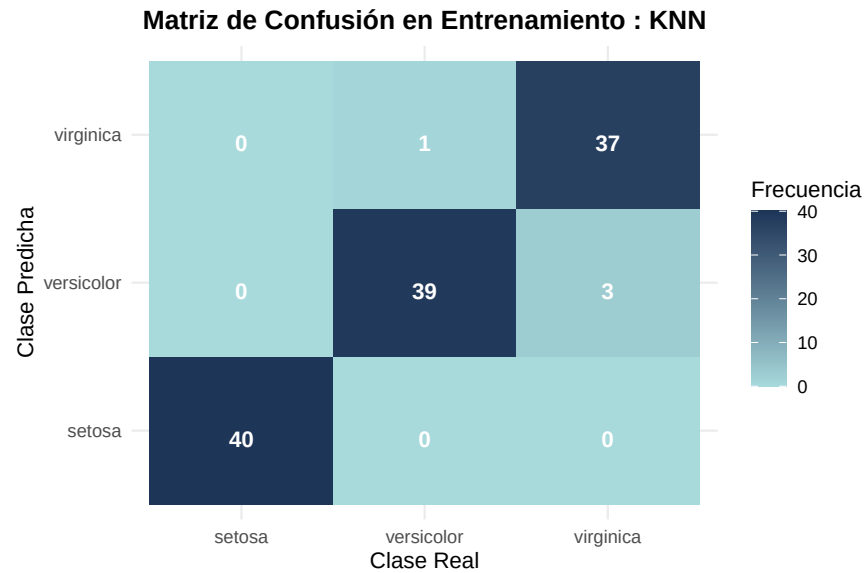
Evaluación del modelo

Al igual que en los modelos anteriores, evaluamos el rendimiento tanto sobre el conjunto de entrenamiento como sobre el conjunto de test. La evaluación en entrenamiento permite detectar si el modelo se ajusta demasiado a los datos utilizados para construirlo, mientras que el conjunto de test proporciona una estimación de su capacidad de generalización.

```
# Predicción sobre el conjunto de entrenamiento
pred_knn_train <- predict(mod_knn, newdata = trainData)

# Matriz de confusión sobre entrenamiento
cm_knn_train <- confusionMatrix(pred_knn_train, trainData$Species)

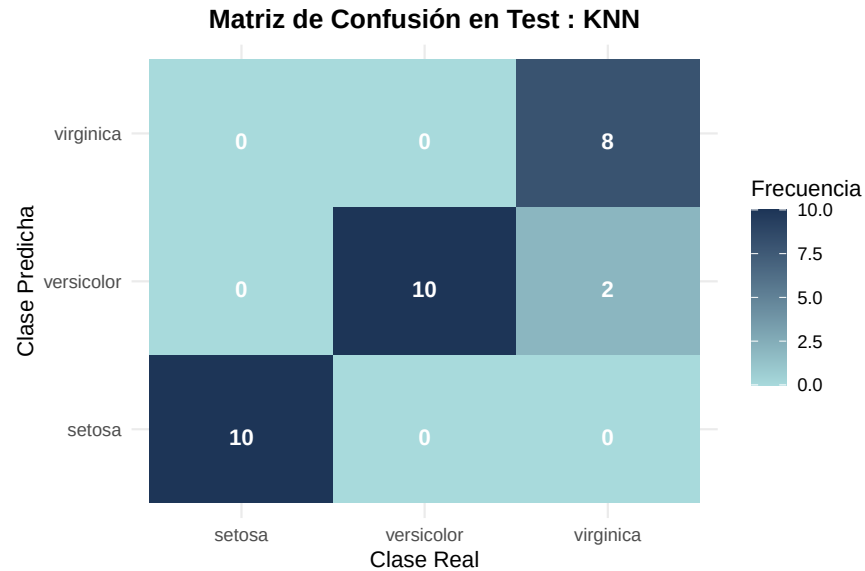
plot_cm(cm_knn_train, "KNN", "Entrenamiento")
```



```
# Predicción sobre el conjunto de test
pred_knn_test <- predict(mod_knn, newdata = testData)

# Matriz de confusión sobre test
cm_knn_test <- confusionMatrix(pred_knn_test, testData$Species)

plot_cm(cm_knn_test, "KNN", "Test")
```



Las matrices de confusión muestran que el modelo KNN obtiene un rendimiento bastante bueno tanto en entrenamiento como en test. En el conjunto de entrenamiento clasifica correctamente 116 de las 120 observaciones, lo que supone una precisión aproximada del 96.7%. En el conjunto de test clasifica correctamente 28 de las 30 observaciones, con una precisión aproximada del 93.3%.

La clase setosa se identifica perfectamente en ambos conjuntos, lo cual era esperable porque suele estar claramente separada de las otras especies. Los errores aparecen únicamente entre versicolor y virginica, que son las dos clases más parecidas entre sí según las variables medidas. En test, las dos observaciones mal

clasificadas corresponden a flores virginica que el modelo predice como versicolor.

En general, los resultados indican que KNN funciona bien para este problema. Además, como el rendimiento en test es similar al de entrenamiento, no parece haber un sobreajuste importante.

Análisis de los resultados

En conjunto, los resultados muestran que KNN funciona bien para el problema de clasificación del conjunto de datos iris. Las matrices de confusión muestran una exactitud alta tanto en entrenamiento, 96.67%, como en test, 93.33%. Además, al ser ambos valores similares, no parece haber un sobreajuste importante. no parece haber un sobreajuste importante, por lo que el modelo obtiene una frontera de decisión suficientemente flexible, pero sin depender en exceso de puntos individuales.

Los errores se concentran entre versicolor y virginica, que son las especies con mayor solapamiento, mientras que setosa se clasifica correctamente al estar claramente separada por las variables del pétalo. Esto también se aprecia en la frontera de decisión, donde setosa ocupa una región diferenciada y las dudas aparecen en la zona de transición entre versicolor y virginica.

En conclusión, KNN ofrece un rendimiento sólido en este problema, combinando una frontera de decisión flexible con una buena capacidad de generalización gracias a la selección de k mediante validación cruzada.

3.4. Máquinas de Vector de Soporte (SVM) con Kernel Radial

Las Máquinas de Vector de Soporte (SVM) constituyen uno de los algoritmos más potentes y versátiles en el ámbito del Aprendizaje Automático. El objetivo principal de una SVM es encontrar el hiperplano óptimo en un espacio N-dimensional que maximice el margen de separación entre las distintas clases.

En problemas donde las clases no son linealmente separables en el espacio original, SVM emplea el conocido “truco del kernel” (*kernel trick*). Este mecanismo proyecta implícitamente los datos de entrada a un espacio de mayor dimensión donde sí es posible trazar un hiperplano lineal separador. En este caso, utilizaremos el kernel de Base Radial (RBF, por sus siglas en inglés), cuya función matemática se define como:

$$K(x, x') = \exp(-\sigma ||x - x'||^2)$$

El comportamiento de este modelo está gobernado principalmente por dos hiperparámetros que optimizaremos mediante validación cruzada:

El parámetro de coste (C): Controla el equilibrio entre lograr un margen amplio y penalizar los errores de clasificación en el conjunto de entrenamiento. Un C alto crea un modelo más estricto (riesgo de sobreajuste), mientras que un C bajo permite un margen más suave (mayor generalización).

El parámetro del kernel (σ o *sigma*): Define la influencia de una única observación. Valores altos de σ hacen que la frontera de decisión sea muy dependiente de los puntos individuales más cercanos, generando fronteras complejas y ajustadas.

Al igual que ocurre con las Redes Neuronales, los algoritmos basados en distancias como SVM son muy sensibles a la escala de las características, por lo que incluiremos un preprocesamiento de estandarización.

Construcción y entrenamiento del modelo

```
# install.packages("kernlab")
set.seed(123)

# Definimos una malla de hiperparámetros para optimizar C y sigma
svmGrid <- expand.grid(
  sigma = c(0.01, 0.1, 0.5, 1), # Amplitud del kernel radial
  C = c(0.1, 1, 10, 100)        # Coste o penalización
)

# Entrenamiento del modelo SVM con Kernel Radial
mod_svm <- train(
  Species ~ .,
  data = trainData,
  method = "svmRadial",
  preProcess = c("center", "scale"),
  tuneGrid = svmGrid,
  trControl = trainControl(method = "cv", number = 10) # 10-fold Cross-Validation
)

# Mostramos los hiperparámetros óptimos seleccionados por validación cruzada
mod_svm$bestTune
#>   sigma    C
#> 4 0.01 100
```

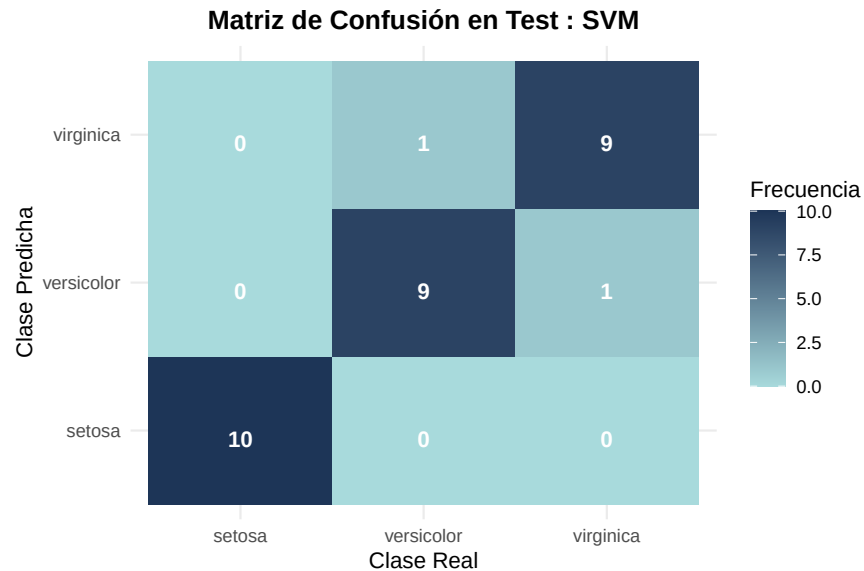

Evaluación del modelo

Una vez ajustados los hiperparámetros óptimos, procedemos a evaluar el modelo sobre nuestro conjunto de test del 20% reservado inicialmente.

```
# Predicción sobre los datos de test
pred_svm <- predict(mod_svm, newdata = testData)

# Generación de la matriz de confusión
cm_svm <- confusionMatrix(pred_svm, testData$Species)

plot_cm(cm_svm, "SVM", "Test")
```



Análisis de los resultados

La Máquina de Vector de Soporte con kernel radial exhibe un rendimiento sobresaliente, alineándose con las métricas observadas en los modelos anteriores. Como era de esperar, la clase setosa se clasifica perfectamente debido a su aislamiento espacial. El aspecto más destacable se observa en el gráfico de la frontera de decisión: a diferencia de la Regresión Logística, cuyas divisiones son estrictamente rectas, el kernel RBF permite que las fronteras se “doblen” de forma no lineal para encapsular la región de solapamiento entre las especies versicolor y virginica. Al haber optimizado los parámetros C y σ , la frontera resultante traza un límite suave y natural entre ambas especies, encontrando un equilibrio idóneo entre el ajuste a los datos de entrenamiento y la capacidad de generalización sin incurrir en un sobreajuste evidente de la frontera hacia puntos ruidosos aislados.

3.5. Gradient Boosting

Gradient Boosting es un método basado en la combinación secuencial de múltiples árboles de decisión poco profundos en general. La idea consiste en construir una sucesión de modelos simples donde cada nuevo árbol intenta corregir los errores cometidos por el conjunto de árboles previamente entrenados.

A diferencia de algoritmos como Random Forest, donde los árboles se generan de forma independiente y en paralelo, Gradient Boosting construye los árboles de manera iterativa. En cada etapa, se ajusta un nuevo árbol sobre los residuos o errores del modelo actual, mejorando progresivamente la capacidad predictiva del conjunto.

Desde un punto de vista matemático, el modelo final puede expresarse como una suma de árboles de decisión:

$$F_M(x) = \sum_{m=1}^M \gamma_m h_m(x),$$

donde $h_m(x)$ representa el árbol construido en la iteración m , γ_m es el peso asociado a dicho árbol y M es el número total de árboles que forman el modelo. Cada nuevo árbol se incorpora siguiendo la dirección del gradiente negativo de una función de pérdida, de ahí el término *Gradient Boosting*.

El comportamiento del algoritmo está determinado principalmente por tres hiperparámetros que optimizaremos más adelante:

Número de árboles (n.trees): controla cuántos árboles se combinan en el modelo final. Un número reducido puede producir infraajuste, mientras que un número excesivamente elevado puede incrementar el riesgo de sobreajuste.

Profundidad de interacción (interaction.depth): determina la complejidad máxima de cada árbol individual. Valores pequeños generan fronteras de decisión más simples, mientras que profundidades elevadas permiten capturar interacciones complejas entre las variables.

Tasa de aprendizaje (shrinkage): regula la contribución de cada nuevo árbol al modelo global. Valores pequeños suelen mejorar la capacidad de generalización, aunque requieren un mayor número de árboles para alcanzar un rendimiento óptimo.

Una de las principales ventajas de Gradient Boosting es su capacidad para modelar relaciones no lineales entre las variables explicativas y la variable objetivo. Además, al estar basado en árboles de decisión, no requiere asumir relaciones lineales entre las variables ni es especialmente sensible a diferencias de escala entre los predictores. Esto lo convierte en una alternativa flexible y potente para problemas de clasificación como el conjunto de datos iris que usamos.

Construcción y entrenamiento del modelo

Para mantener una metodología homogénea con el resto de modelos analizados, utilizaremos la función `train()` del paquete `caret`, que permite automatizar el proceso de ajuste y selección de hiperparámetros mediante validación cruzada.

Con el objetivo de encontrar la configuración más adecuada para el conjunto de datos `iris`, definimos un conjunto de posibles valores para los hiperparámetros anteriormente comentados. La selección de la mejor combinación se realizará mediante validación cruzada de 10 particiones (*10-fold Cross Validation*).

El uso de validación cruzada permite estimar el rendimiento esperado del modelo sobre datos no observados durante el entrenamiento, reduciendo la dependencia de una única partición de los datos y proporcionando una selección más robusta de los hiperparámetros.

```
set.seed(123)
```

```
gbmGrid <- expand.grid(
```

```

interaction.depth = c(1,2,3),
n.trees = c(10,25,50,100),
shrinkage = c(0.001,0.01,0.1),
n.minobsinnode = 10
)

mod_gbm <- train(
  Species ~ .,
  data = trainData,
  method = "gbm",
  tuneGrid = gbmGrid,
  trControl = trainControl(method = "cv", number = 10),
  verbose = FALSE
)

mod_gbm$bestTune
#>      n.trees interaction.depth shrinkage n.minobsinnode
#> 33      10           3         0.1           10

```

Evaluación del modelo

Una vez seleccionada la combinación óptima de hiperparámetros mediante validación cruzada, procedemos a evaluar el rendimiento del modelo tanto sobre el conjunto de entrenamiento como sobre el conjunto de test. Esta comparación permitirá analizar la capacidad de generalización del modelo y detectar posibles indicios de sobreajuste.

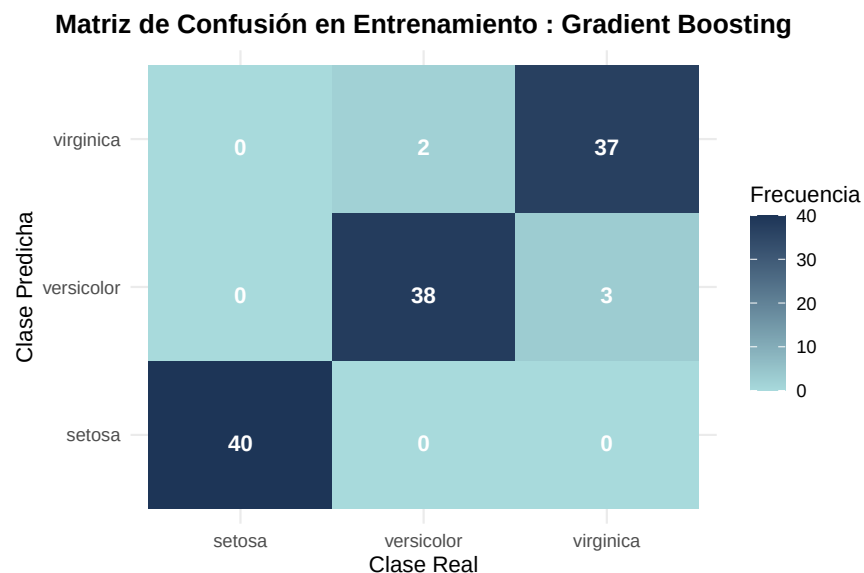
```

# Predicción sobre el conjunto de entrenamiento
pred_gbm_train <- predict(mod_gbm, newdata = trainData)

# Matriz de confusión sobre entrenamiento
cm_gbm_train <- confusionMatrix(pred_gbm_train, trainData$Species)

plot_cm(cm_gbm_train, "Gradient Boosting", "Entrenamiento")

```



```

# Predicción sobre el conjunto de test

```

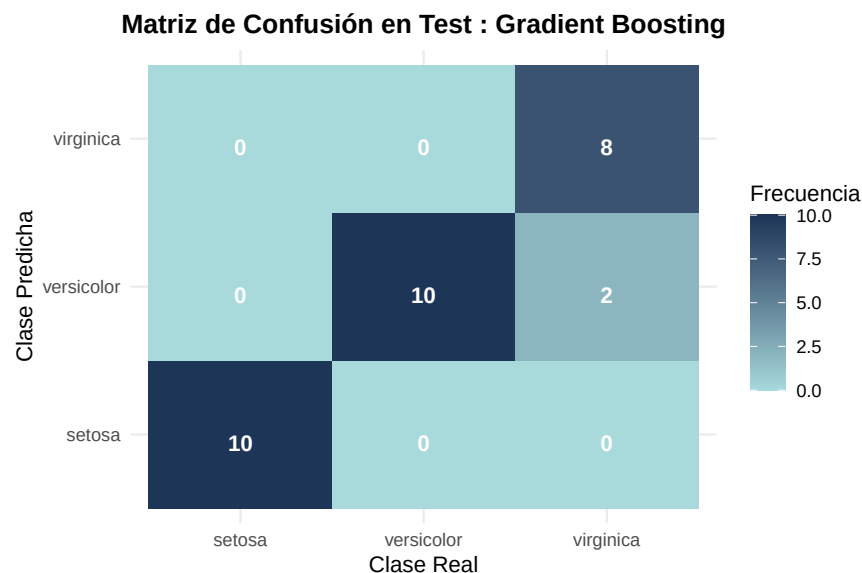
```

pred_gbm_test <- predict(mod_gbm, newdata = testData)

# Matriz de confusión sobre test
cm_gbm_test <- confusionMatrix(pred_gbm_test, testData$Species)

plot_cm(cm_gbm_test, "Gradient Boosting", "Test")

```



Las matrices de confusión muestran que Gradient Boosting obtiene un rendimiento muy elevado sobre el conjunto de entrenamiento. El modelo clasifica correctamente 115 de las 120 observaciones, alcanzando una exactitud aproximada del 95,83%.

La especie setosa se identifica perfectamente en todos los casos, lo que refleja la clara separación existente entre esta clase y las demás. Los únicos errores aparecen entre las especies versicolor y virginica, que presentan características morfológicas más similares y, por tanto, una mayor zona de solapamiento.

Estos resultados indican que el modelo es capaz de capturar los patrones presentes en los datos de entrenamiento sin necesidad de recurrir a fronteras de decisión excesivamente complejas.

Análisis de los resultados

Gradient Boosting obtiene un rendimiento muy elevado tanto en entrenamiento como en test. En el conjunto de entrenamiento clasifica correctamente 115 de las 120 observaciones, alcanzando una exactitud del 95,83%. Por su parte, en el conjunto de test clasifica correctamente 28 de las 30 observaciones, lo que supone una exactitud del 93,33%.

La especie setosa se identifica correctamente en todos los casos, tanto en entrenamiento como en test. Los errores de clasificación se concentran exclusivamente entre las especies versicolor y virginica, que presentan características morfológicas más similares y una mayor zona de solapamiento en el espacio de variables.

La diferencia de rendimiento entre entrenamiento y test es reducida, lo que sugiere que el modelo presenta una buena capacidad de generalización y no muestra indicios claros de sobreajuste. Esto indica que la estrategia de validación cruzada utilizada para seleccionar los hiperparámetros ha permitido obtener un equilibrio adecuado entre complejidad y capacidad predictiva.

La frontera de decisión confirma esta interpretación. Se observa una separación clara de la especie setosa, mientras que la región que separa versicolor y virginica presenta una estructura más compleja. La forma escalonada de las fronteras refleja la naturaleza de los árboles de decisión que componen el modelo, permitiendo capturar relaciones no lineales sin necesidad de asumir una forma funcional específica.

En conjunto, los resultados muestran que Gradient Boosting constituye una alternativa potente y flexible para este problema de clasificación, ofreciendo una elevada precisión y una buena capacidad de generalización sobre datos no observados.

3.6. Red Neuronal (Multilayer Perceptron)

Como último modelo predictivo, implementamos una Red Neuronal Artificial (*Artificial Neural Network*, ANN) mediante el paquete `nnet`. Específicamente, construimos un Perceptrón Multicapa (MLP) del tipo *feed-forward* con una única capa oculta.

A diferencia de los métodos de particionamiento recursivo (como los árboles) o basados en distancias (como KNN), la red neuronal busca aproximar la función de clasificación mediante una combinación lineal de transformaciones no lineales. En nuestro caso, la capa de salida utiliza una transformación logística multinomial (*softmax*) para emitir una distribución de probabilidad sobre las tres posibles especies botánicas.

Matemáticamente, la capa oculta realiza una transformación no lineal del espacio de entrada original $\mathbb{R}^4$. Si denotamos el vector de características de entrada como X y la matriz de pesos sinápticos como W , las neuronas aplican una función de activación logística $f(x) = \frac{1}{1+e^{-x}}$ sobre la combinación lineal $W^T X$. Esto proyecta los datos geométricos de las flores en un nuevo espacio multidimensional donde las clases se vuelven linealmente separables para la capa de salida.

Cabe destacar que las redes neuronales son extremadamente sensibles a la escala de las variables de entrada debido a la forma en que se actualizan los gradientes durante la optimización. Por ello, delegaremos en `caret` la tarea de preprocesar los datos (`center` y `scale`) para que todas las variables tengan media nula y varianza unitaria. Además, para evitar el sobreajuste (*overfitting*) en un conjunto de datos relativamente pequeño, aplicaremos una penalización sobre la norma de los pesos de la red, conocida como regularización de Tikhonov o *weight decay* (λ). A través de `caret`, realizaremos una búsqueda en malla para encontrar la combinación óptima de neuronas en la capa oculta (`size`) y el factor de decaimiento (`decay`).

```
# Cargamos la librería para poder dibujar la red neuronal
#install.packages("NeuralNetTools")
library(NeuralNetTools)

set.seed(123)

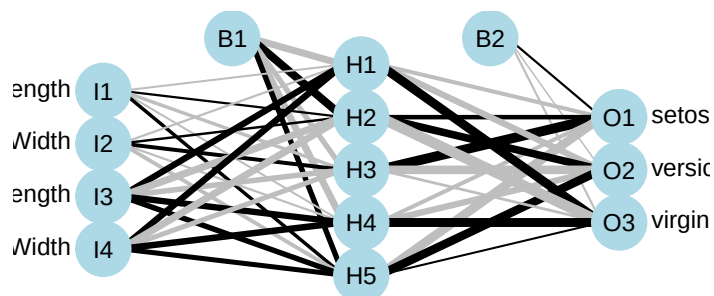
# Definimos una malla de hiperparámetros para optimizar la red
nnetGrid <- expand.grid(
  size = c(3, 5, 8),          # Número de neuronas en la capa oculta
  decay = c(0.1, 0.01, 0.001) # Parámetro de penalización L2 (weight decay)
)

# Entrenamiento del modelo
mod_nnet <- train(
  Species ~ .,
  data = trainData,
  method = "nnet",
  preProcess = c("center", "scale"), # Estandarizamos las variables internamente
  tuneGrid = nnetGrid,
  trace = FALSE,                    # Suprimimos el output iterativo del optimizador
  MaxNWts = 500                     # Límite de pesos para la convergencia
)

# Mostramos los hiperparámetros óptimos seleccionados por validación cruzada
mod_nnet$bestTune
#>   size decay
#> 6     5   0.1
```

Para comprender mejor cómo el modelo procesa la información y abrir la “caja negra” del algoritmo, visualizamos la topología final de la red. Cada línea representa un peso sináptico w_{ij} . El grosor y color de estas conexiones nos indica qué mediciones de la flor están activando con mayor intensidad las neuronas de la capa oculta, actuando como extractoras de características geométricas.

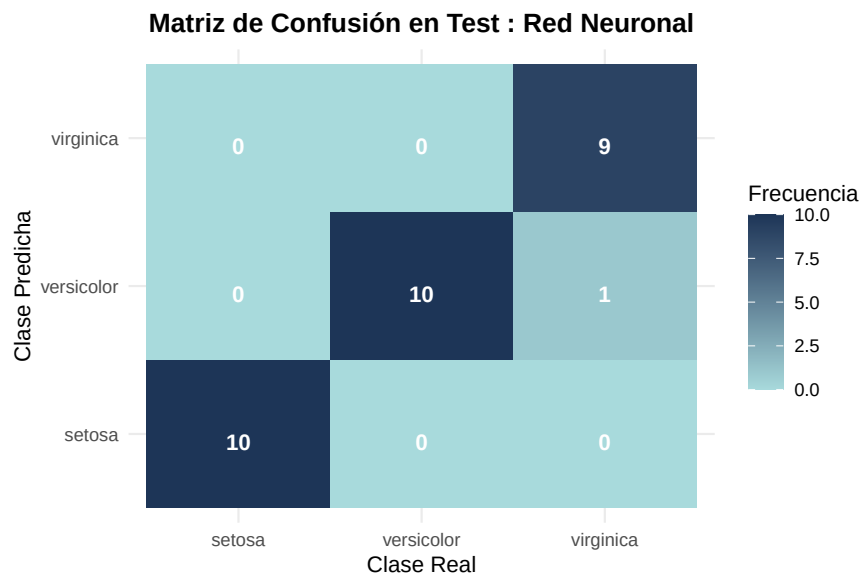
```
# Visualización de la topología de la red neuronal
plotnet(mod_nnet$finalModel, main = "Arquitectura del Perceptrón Multicapa")
```



Una vez que caret ha determinado la arquitectura óptima, procedemos a evaluar el rendimiento de la red neuronal sobre el conjunto de test independiente.

```
# Predicción sobre los datos de validación
pred_nnet <- predict(mod_nnet, newdata = testData)

# Generación de la matriz de confusión
cm_nnet <- confusionMatrix(pred_nnet, testData$Species)
plot_cm(cm_nnet, "Red Neuronal", "Test")
```



Análisis de los resultados

La evaluación general de la red neuronal confirma una capacidad de clasificación excepcional. Observamos que el modelo logra una clasificación perfecta para la especie setosa, lo cual es consistente tanto en el mapa

de calor como en la separación total que se evidencia en el gráfico de la frontera de decisión bidimensional.

El modelo muestra una dificultad mínima en la distinción entre versicolor y virginica (un único error de clasificación en la matriz de confusión). Esto es teóricamente coherente dada la superposición natural de estas dos especies. Como demuestra el gráfico de proyección bidimensional, aunque la red neuronal tiene la capacidad teórica de generar fronteras no lineales, ha optado por trazar divisiones casi rectilíneas. Este fenómeno ilustra perfectamente el impacto del parámetro de regularización (weight decay): al penalizar los pesos grandes para evitar el sobreajuste, las funciones de activación operan en su régimen lineal. La red concluye que una frontera simple es la solución óptima y más generalizable para este subespacio 2D, evitando crear curvas innecesarias ante datos que ya presentan una clara separabilidad lineal.

4. Análisis Visual Comparativo de Fronteras de Decisión

Para comprender a nivel espacial cómo cada algoritmo fragmenta el espacio de características, hemos entrenado modelos auxiliares bidimensionales utilizando exclusivamente `Petal.Length` y `Petal.Width`.

```
set.seed(123)

# Dataset reducido y malla espacial común
iris_2d <- trainData[, c("Petal.Length", "Petal.Width", "Species")]
grid_base <- expand.grid(
  Petal.Length = seq(min(iris$Petal.Length), max(iris$Petal.Length), length.out = 150),
  Petal.Width = seq(min(iris$Petal.Width), max(iris$Petal.Width), length.out = 150)
)

# Entrenamiento de modelos 2D (usando los hiperparámetros óptimos)
mod_logit_2d <- train(Species ~ ., data = iris_2d, method = "multinom", trace = FALSE)

mod_rf_2d <- train(Species ~ ., data = iris_2d, method = "rf",
  tuneGrid = data.frame(mtry = mod_rf$bestTune$mtry),
  trControl = trainControl(method="none"))

mod_knn_2d <- train(Species ~ ., data = iris_2d, method = "knn",
  preProcess = c("center", "scale"),
  tuneGrid = data.frame(k = mod_knn$bestTune$k),
  trControl = trainControl(method="none"))

mod_svm_2d <- train(Species ~ ., data = iris_2d, method = "svmRadial",
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(sigma = mod_svm$bestTune$sigma, C = mod_svm$bestTune$C),
  trControl = trainControl(method="none"))

mod_gbm_2d <- train(Species ~ ., data = iris_2d, method = "gbm", verbose = FALSE,
  tuneGrid = mod_gbm$bestTune,
  trControl = trainControl(method="none"))

mod_nnet_2d <- train(Species ~ ., data = iris_2d, method = "nnet", trace = FALSE,
  preProcess = c("center", "scale"),
  tuneGrid = mod_nnet$bestTune,
  trControl = trainControl(method="none"))

# Función auxiliar para extraer predicciones y empaquetarlas
get_preds <- function(model, name) {
  temp_grid <- grid_base
  temp_grid$Species <- predict(model, newdata = grid_base)
  temp_grid$Modelo <- name
  return(temp_grid)
}

# Unimos todas las predicciones en un solo Data Frame
preds_all <- rbind(
  get_preds(mod_logit_2d, "Regresión Logística"),
  get_preds(mod_rf_2d, "Random Forest"),
  get_preds(mod_knn_2d, "KNN"),
  get_preds(mod_svm_2d, "SVM Radial"),
```

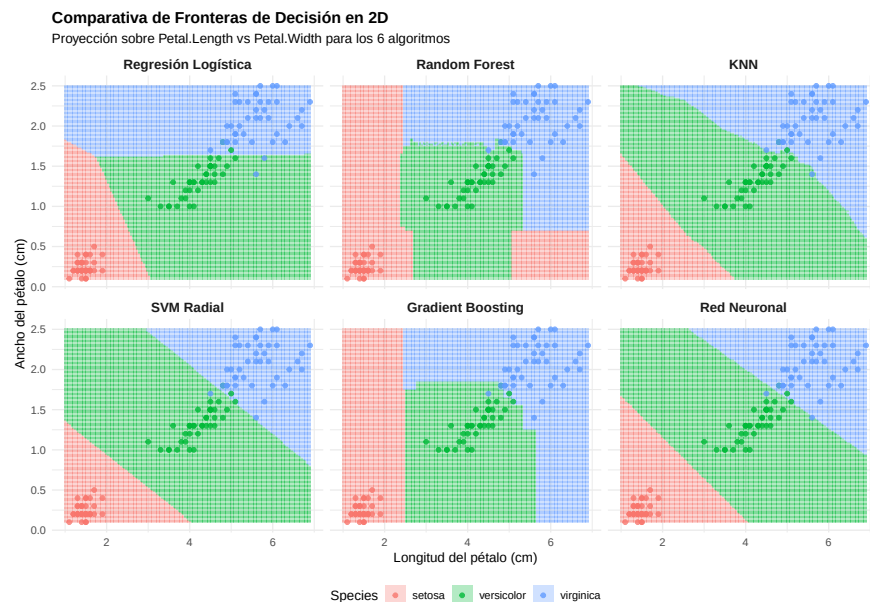
```

get_preds(mod_gbm_2d, "Gradient Boosting"),
get_preds(mod_nnet_2d, "Red Neuronal")
)

# Aseguramos el orden en el que se dibujarán los paneles
preds_all$Modelo <- factor(preds_all$Modelo,
                           levels = c("Regresión Logística", "Random Forest", "KNN",
                                       "SVM Radial", "Gradient Boosting", "Red Neuronal"))

ggplot() +
  geom_tile(data = preds_all, aes(x = Petal.Length, y = Petal.Width, fill = Species), alpha = 0.3) +
  geom_point(data = iris_2d, aes(x = Petal.Length, y = Petal.Width, color = Species),
            size = 1.5, alpha = 0.8) +
  scale_fill_manual(values = c("#F8766D", "#00BA38", "#619CFF")) +
  scale_color_manual(values = c("#F8766D", "#00BA38", "#619CFF")) +
  facet_wrap(~ Modelo, ncol = 3) +
  theme_minimal() +
  labs(title = "Comparativa de Fronteras de Decisión en 2D",
       subtitle = "Proyección sobre Petal.Length vs Petal.Width para los 6 algoritmos",
       x = "Longitud del pétalo (cm)",
       y = "Ancho del pétalo (cm)") +
  theme(plot.title = element_text(face = "bold"),
        strip.text = element_text(face = "bold", size = 11),
        legend.position = "bottom")

```



Al observar la cuadrícula de proyecciones bidimensionales generada, podemos apreciar claramente cómo la “filosofía” matemática base de cada algoritmo se traduce de forma visual en la manera de dividir el plano de características:

- **Regresión Logística:** Como modelo paramétrico lineal, muestra fronteras estrictamente rectas, trazando cortes geométricos limpios. A pesar de carecer de la flexibilidad necesaria para curvarse alrededor de los datos, su acierto empírico demuestra que una separación lineal simple es notablemente efectiva para este problema.
- **Modelos basados en Árboles (Random Forest y Gradient Boosting):** Ambos evidencian el

comportamiento de partición recursiva. En lugar de líneas diagonales o curvas, generan fronteras ortogonales a los ejes (en forma de “escalera” o bloques). Mientras que Random Forest tiende a aislar subregiones más específicas (pudiendo bordear el ruido local), Gradient Boosting —gracias al control de la tasa de aprendizaje o *shrinkage*— presenta bloques y transiciones más uniformes.

- **K-Nearest Neighbors (KNN):** Al ser un modelo basado puramente en distancias locales, proporciona regiones sumamente irregulares y adaptadas al enjambre de puntos más cercanos. Exhibe una topología muy flexible y fragmentada, guiada exclusivamente por la densidad de los clústeres en zonas de solapamiento.
- **Máquinas de Vector de Soporte (SVM Radial):** El uso del *kernel trick* transforma las rígidas separaciones lineales en formas curvas y suaves. Logra “envolver” eficazmente la difusa zona de transición entre *versicolor* y *virginica*, encontrando un hiperplano de separación de margen máximo sin que las fronteras se desvirtúen intentando atrapar valores atípicos (*outliers*).
- **Red Neuronal (MLP):** Resulta un caso de estudio muy interesante. Aunque la arquitectura del perceptrón multicapa posee la capacidad teórica para mapear límites altamente no lineales, la aplicación deliberada de la regularización (*weight decay*) restringe fuertemente su complejidad gráfica. El optimizador concluye que trazar divisiones casi rectilíneas es la solución más robusta y generalizable para evitar el sobreajuste en este espacio de dos dimensiones.

5. Discusión y Conclusiones

```
# Predicciones en entrenamiento faltantes (SVM y Red Neuronal)
pred_svm_train <- predict(mod_svm, newdata = trainData)
cm_svm_train <- confusionMatrix(pred_svm_train, trainData$Species)

pred_nnet_train <- predict(mod_nnet, newdata = trainData)
cm_nnet_train <- confusionMatrix(pred_nnet_train, trainData$Species)

# Creación del Data Frame con las métricas
resultados_modelos <- data.frame(
  Modelo = c(
    "Regresión Logística",
    "Random Forest",
    "K-Nearest Neighbors (KNN)",
    "SVM (Kernel Radial)",
    "Gradient Boosting",
    "Red Neuronal (MLP)"
  ),
  Accuracy_Train = c(
    cm_logit_train$overall['Accuracy'],
    cm_rf_train$overall['Accuracy'],
    cm_knn_train$overall['Accuracy'],
    cm_svm_train$overall['Accuracy'],
    cm_gbm_train$overall['Accuracy'],
    cm_nnet_train$overall['Accuracy']
  ),
  Accuracy_Test = c(
    cm_logit_test$overall['Accuracy'],
    cm_rf_test$overall['Accuracy'],
    cm_knn_test$overall['Accuracy'],
    cm_svm$overall['Accuracy'], # En tu código se llamaba cm_svm
    cm_gbm_test$overall['Accuracy'],
    cm_nnet$overall['Accuracy'] # En tu código se llamaba cm_nnet
  )
)

# Formatear los números
resultados_modelos$Accuracy_Train <- paste0(round(resultados_modelos$Accuracy_Train * 100, 2), "%")
resultados_modelos$Accuracy_Test <- paste0(round(resultados_modelos$Accuracy_Test * 100, 2), "%")

# Generar la tabla
knitr::kable(
  resultados_modelos,
  align = "lcc",
  col.names = c("Modelo Algorítmico", "Exactitud (Entrenamiento)", "Exactitud (Test)"),
  caption = "Comparativa del rendimiento global de los modelos de clasificación"
)
```

Table 1: Comparativa del rendimiento global de los modelos de clasificación

Modelo Algorítmico	Exactitud (Entrenamiento)	Exactitud (Test)
Regresión Logística	98.33%	96.67%
Random Forest	100%	93.33%
K-Nearest Neighbors (KNN)	96.67%	93.33%
SVM (Kernel Radial)	99.17%	93.33%
Gradient Boosting	95.83%	93.33%
Red Neuronal (MLP)	98.33%	96.67%

Después de aplicar y evaluar seis modelos distintos de clasificación sobre el dataset *iris*, hemos comprobado que separar la especie *setosa* no supone ningún problema para ninguno de los algoritmos, ya que sus medidas están muy alejadas del resto. El verdadero reto analítico del trabajo ha sido trazar la frontera entre *versicolor* y *virginica*, ya que sus características geométricas se solapan bastante.

Al comparar cómo cada modelo afronta este problema, hemos visto enfoques muy distintos que llegan a resultados similares:

- **La Regresión Logística Multinomial** ha funcionado como un modelo base excelente. Aunque solo traza divisiones rectas, su alto porcentaje de acierto demuestra que a veces no hace falta un modelo extremadamente complejo para obtener buenos resultados.
- **Los métodos basados en distancias (KNN y SVM Radial)** nos han obligado a escalar los datos previamente. KNN clasifica fijándose únicamente en los datos más cercanos, mientras que SVM (gracias al kernel radial) consigue adaptar una frontera curva muy suave justo en la zona donde las especies se mezclan.
- **Los modelos basados en árboles (Random Forest y Gradient Boosting)** dividen el espacio con cortes rectos (formando una especie de escalera). Aunque operan de forma distinta —Random Forest hace la media de muchos árboles independientes y Gradient Boosting mejora los errores paso a paso—, ambos han dado resultados sobresalientes sin necesidad de escalar las variables geométricas.
- Por último, **la Red Neuronal** es el modelo con mayor capacidad para ajustarse a formas complejas. Sin embargo, hemos visto que gracias a la regularización que le aplicamos (*weight decay*), el modelo decidió trazar fronteras casi rectas, evitando sobreajustarse a los datos de entrenamiento para poder generalizar mejor.

A nivel de programación, desarrollar todo este análisis comparativo habría sido mucho más largo y propenso a errores sin el uso del paquete *caret*. Esta librería nos ha permitido unificar el preprocesamiento, la validación cruzada y las métricas de evaluación para modelos que por debajo usan paquetes muy diferentes (*nnet*, *kernlab*, *randomForest*, etc.). Esto nos garantiza que la comparación entre todos los algoritmos haya sido justa y en igualdad de condiciones.

Como conclusión final, este trabajo nos demuestra que en un dataset de estas características, aplicar algoritmos muy complejos no supone una mejora drástica frente a los modelos clásicos. Aunque SVM o las Redes Neuronales tienen mucha más flexibilidad, opciones más sencillas como la Regresión Logística o KNN ofrecen resultados prácticamente idénticos. Por tanto, para este caso concreto, la mejor decisión sería optar por los modelos más simples, ya que su coste computacional es mucho menor y son más fáciles de interpretar.